

PART 2

Control Flow

Chapters 4–6: Branching, Loops, and Tool Calling

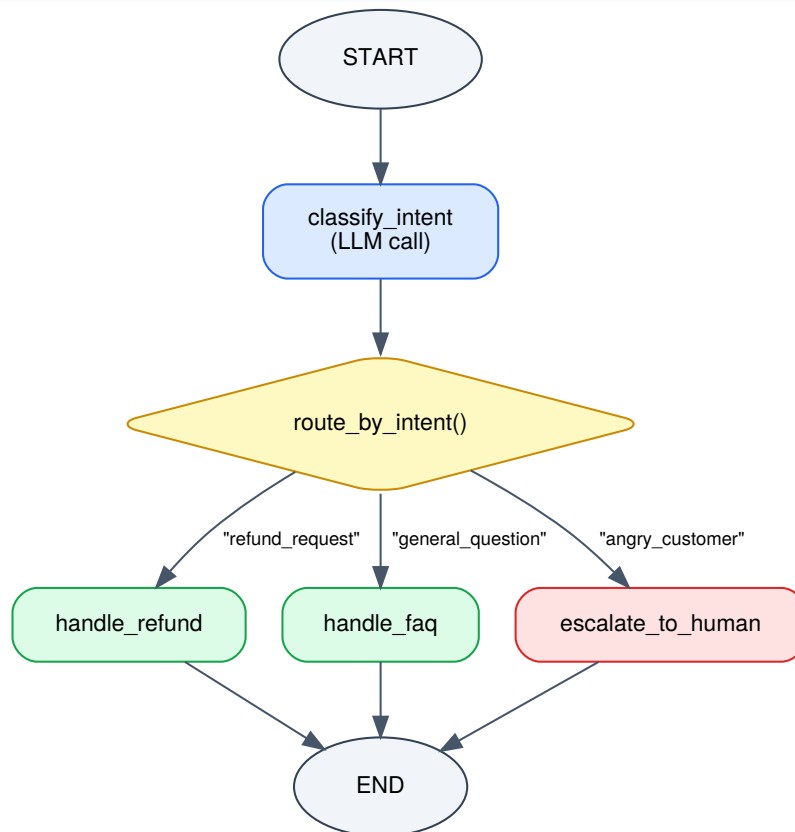
Conditional Edges & Branching

4.1 Conceptual Overview

Chapter 2 introduced conditional edges in the abstract: a function that looks at state and returns the name of the next node. But the real skill is deciding *where* in your graph a branch belongs, and *what* the routing function should actually inspect. Let's ground this in a system you'd genuinely build at a company: a **customer-support triage agent** for an e-commerce store.

Running example — Support Triage Agent: A customer writes in: "This is the third time my order hasn't arrived, I want my money back NOW." The agent needs to decide, before doing anything else, which of three very different workflows applies: process a refund, answer a general FAQ, or hand off to a human because the customer is upset. Each path needs different tools, different tone, and in the escalation case, no further automation at all.

This is exactly what a conditional edge is for: one upstream node (`classify_intent`) produces a signal, and a routing function reads that signal and sends execution down one of several distinct paths. Note that the branches aren't just "different prompts" — they're structurally different subgraphs of work (a refund path touches a payments API; an FAQ path touches a knowledge base; an escalation path touches neither and instead writes a ticket for a human).



Customer-support triage agent: one classify node, three-way conditional branch

4.1.1 Where the classification signal comes from

In practice you have two options for `classify_intent`: ask the LLM to output a structured label (e.g. via `with_structured_output` against a Pydantic enum), or let the main agent LLM emit a tool call whose *name* itself signals intent (e.g. `initiate_refund` vs `escalate`). The triage example below uses the first approach because it's more predictable for a decision that must be reliable — you don't want "refund" and "escalate" confused due to a vague tool description.

Key idea: A conditional edge's routing function should be *cheap and deterministic* — it reads state and returns a string, nothing more. All the actual work (calling the payments API, drafting an FAQ answer) belongs inside the node it routes *to*, not inside the router itself.

4.1.2 Multi-way branching, not just if/else

Unlike a simple if/else, `add_conditional_edges` naturally scales to N branches — the routing function just returns one of N possible node names. This is why it fits triage-style problems so well: a real support inbox doesn't have two categories, it has a dozen.

4.2 Implementation

```
from typing import TypedDict, Annotated, Literal
from pydantic import BaseModel
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI

class Intent(BaseModel):
    label: Literal["refund_request", "general_question", "angry_customer"]

class SupportState(TypedDict):
    messages: Annotated[list, add_messages]
    intent: str

classifier_llm = ChatOpenAI(model="gpt-4o-mini").with_structured_output(Intent)

def classify_intent(state: SupportState) -> dict:
    user_msg = state["messages"][-1].content
    result = classifier_llm.invoke(
        f"Classify this customer message: {user_msg}"
    )
    return {"intent": result.label}

def handle_refund(state: SupportState) -> dict:
    # In a real system: look up order, call payments API, etc.
    return {"messages": [("assistant", "I've started your refund – 3-5 business days.")]}

def handle_faq(state: SupportState) -> dict:
    return {"messages": [("assistant", "Here's how our shipping policy works...")]}

def escalate_to_human(state: SupportState) -> dict:
    # In a real system: create a ticket in Zendesk/Freshdesk, notify a human
    return {"messages": [("assistant", "I'm connecting you with a support specialist now.")]}

def route_by_intent(state: SupportState) -> str:
    return {
        "refund_request": "handle_refund",
        "general_question": "handle_faq",
        "angry_customer": "escalate_to_human",
    }[state["intent"]]

builder = StateGraph(SupportState)
builder.add_node("classify_intent", classify_intent)
builder.add_node("handle_refund", handle_refund)
builder.add_node("handle_faq", handle_faq)
builder.add_node("escalate_to_human", escalate_to_human)

builder.add_edge(START, "classify_intent")
builder.add_conditional_edges("classify_intent", route_by_intent)
builder.add_edge("handle_refund", END)
builder.add_edge("handle_faq", END)
builder.add_edge("escalate_to_human", END)

graph = builder.compile()

graph.invoke({
    "messages": [("user", "This is the third time my order hasn't arrived, I want my money back NOW.")]
})
# -> classify_intent labels this "angry_customer" (tone signals override the
# literal refund request), so it routes to escalate_to_human, not handle_refund.
```

Note: notice the routing dict pattern — `route_by_intent` maps a label to a node name using a plain dict lookup instead of a chain of `if/elif`. This scales cleanly as you add more intents and is easy to unit-test in isolation from the graph.

4.3 Interview Questions

Q1. In the support-triage example, why does the message "I want my money back NOW" get routed to

escalation instead of the refund handler?

A: The classifier isn't doing keyword matching — it's an LLM call reasoning over the full message, including tone ("third time," capitalized "NOW", frustration). It judges the emotional state as the dominant signal over the literal refund request, and the routing function trusts whatever label the classifier returns. This illustrates that conditional-edge quality is only as good as the signal feeding it — if you wanted refund requests to always win regardless of tone, you'd need to change the classifier's prompt/schema, not the graph.

Q2. Why should the routing function itself stay "cheap and deterministic" rather than doing real work?

A: Routing functions run on every pass through that point in the graph and are meant to be a fast decision, not a side-effecting operation. Mixing real work (API calls, LLM calls) into the router makes the graph's control flow harder to reason about, harder to test (you can't unit test routing logic without triggering side effects), and harder to retry safely if a checkpoint resumes the graph mid-run.

Q3. How does `add_conditional_edges` handle more than two branches, compared to a plain Python if/else?

A: The routing function can return any of N node names — LangGraph doesn't restrict conditional edges to binary choices. This maps naturally to a dict lookup (label → node name) rather than a growing if/elif chain, which is both more explicit and easier to extend as new categories are added.

Q4. Why use `with_structured_output` with a constrained `Literal` type for the classifier instead of asking the model to "reply with one word"?

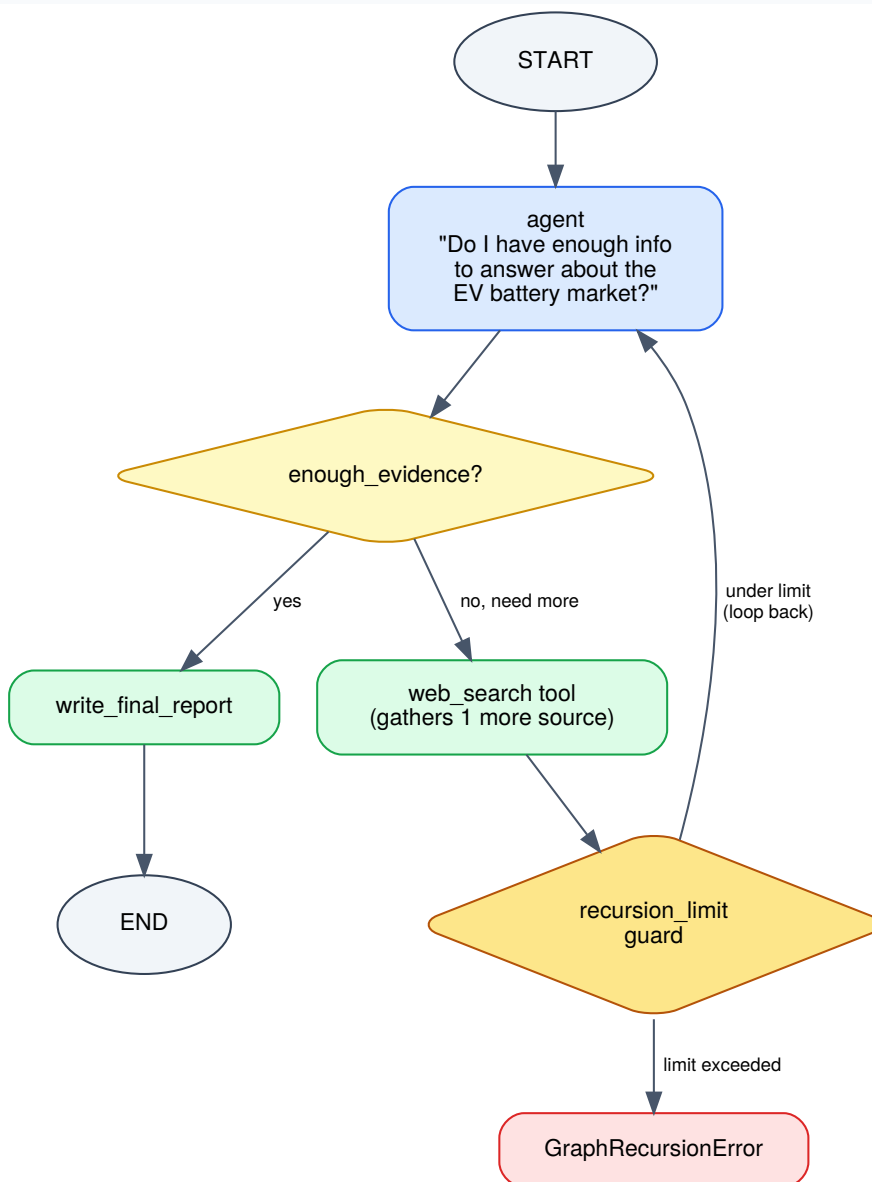
A: A free-text "reply with one word" instruction is not guaranteed — the model might say "Refund Request" with different casing/wording, or add an explanation despite instructions. A structured output schema with a `Literal` type constrains the model's output to one of the exact enum values, so `route_by_intent`'s dict lookup can never `KeyError` on an unexpected label.

Cycles & Reasoning Loops

5.1 Conceptual Overview

Branching (Chapter 4) answers "which path?" A cycle answers a different question: "how many times?" — and crucially, the agent itself decides the answer, not your code. This is the single biggest structural difference between agentic systems and traditional pipelines.

Running example — Research Agent: You ask an agent to write a report on "the current state of the EV battery market." One web search is never enough — the agent needs to search, read what it found, judge whether it has enough evidence, and if not, search again with a refined query. It might take 2 iterations for a narrow question, or 8 for a broad one. You cannot hard-code the number of loop iterations; the agent has to decide when it's done.



A research agent's reasoning loop: it decides for itself how many search iterations it needs, bounded by a recursion limit

5.1.1 The two things every reasoning loop needs

- **A termination condition** the agent controls — some node/router that can genuinely return "done," based on the agent's own judgment of the state (e.g., "I now have 3 credible sources").
- **A hard ceiling you control** — because LLM judgment about "am I done?" is fallible. LangGraph enforces this automatically via a **recursion limit**: the maximum number of *super-steps* (roughly, node executions) a single `.invoke()` call may take before it raises `GraphRecursionError`.

Key idea: Never ship a cyclic graph without a recursion limit in mind. An agent that talks itself into "I need one more source" indefinitely will otherwise loop until you run out of money on API calls, not until it errors out gracefully.

5.1.2 Why loop-back edges are just... edges

There's nothing structurally special about a "loop" in LangGraph — it's simply an edge whose target is a node that already appears earlier in the execution path. `tools -> agent` from Chapter 3 was already a cycle. What's different in this chapter is that the loop is now driven by an explicit evidence-sufficiency check the agent reasons about, not just "did the model request a tool."

5.2 Implementation

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

@tool
def web_search(query: str) -> str:
    """Search the web and return a short snippet of findings."""
    ... # real implementation would call a search API
    return f"Found one relevant source about: {query}"

class ResearchState(TypedDict):
    messages: Annotated[list, add_messages]
    sources_found: int

llm = ChatOpenAI(model="gpt-4o").bind_tools([web_search])

def agent_node(state: ResearchState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

def run_search(state: ResearchState) -> dict:
    last_call = state["messages"][-1].tool_calls[0]
    result = web_search.invoke(last_call["args"])
    return {
        "sources_found": state["sources_found"] + 1,
        "messages": [("tool", result)],
    }

def enough_evidence(state: ResearchState) -> str:
    last_message = state["messages"][-1]
    if not last_message.tool_calls:
        return "write_final_report"          # model itself said "I'm ready"
    if state["sources_found"] >= 5:
        return "write_final_report"          # our own safety ceiling
    return "run_search"

def write_final_report(state: ResearchState) -> dict:
    response = llm.invoke(
        state["messages"] + [("user", "Write the final report now.")]
    )
    return {"messages": [response]}

builder = StateGraph(ResearchState)
builder.add_node("agent", agent_node)
builder.add_node("run_search", run_search)
builder.add_node("write_final_report", write_final_report)

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", enough_evidence)
builder.add_edge("run_search", "agent")
builder.add_edge("write_final_report", END)

graph = builder.compile()

# Global safety net: cap total super-steps for this run
result = graph.invoke(
    {"messages": [("user", "Research the EV battery market.)"], "sources_found": 0},
    {"recursion_limit": 25},
)
```

Note: This example uses *two* stopping conditions layered together: the model's own judgment ("no more tool calls") *and* a hard cap (`sources_found >= 5`) enforced in plain Python, independent of what the model decides. The `recursion_limit` passed at invoke time is a third, coarser safety net on top of both. In production agents, you generally want all three layers — trusting the model alone is how runaway loops and runaway bills happen.

5.3 Interview Questions

Q1. In the research agent, why are there two separate stopping mechanisms (`sources_found >= 5` and the model's own tool-call decision) instead of just trusting the model to stop?

A: The model's judgment about "do I have enough evidence" is a soft signal that can be wrong — a model might keep requesting searches indefinitely for an ambiguous or very broad question, or conversely stop too early. A hard, code-enforced ceiling (`sources_found >= 5`) guarantees termination regardless of model behavior, protecting against runaway loops and unbounded API cost, while the model's own judgment still lets it stop early when it genuinely has enough.

Q2. What is a "super-step" in the context of LangGraph's recursion limit, and what happens when the limit is hit?

A: A super-step is roughly one round of node execution in the graph (each time control passes through a node). The `recursion_limit` config caps how many super-steps a single `.invoke()` / `.stream()` call may execute; exceeding it raises a `GraphRecursionError` rather than looping forever, giving you a controlled failure instead of a silent runaway process.

Q3. Structurally, what makes an edge a "loop" versus a normal forward edge — is there a special API for it?

A: No special API — a loop is simply an edge (normal or conditional) whose target node has already executed earlier in the same run, such as `run_search -> agent` here. LangGraph doesn't distinguish loop-back edges from any other edge; whether a graph is cyclic is just a property of how you wired the edges.

Q4. Why is a fixed number of iterations (e.g., "always search exactly 3 times") usually the wrong design for an agentic research task?

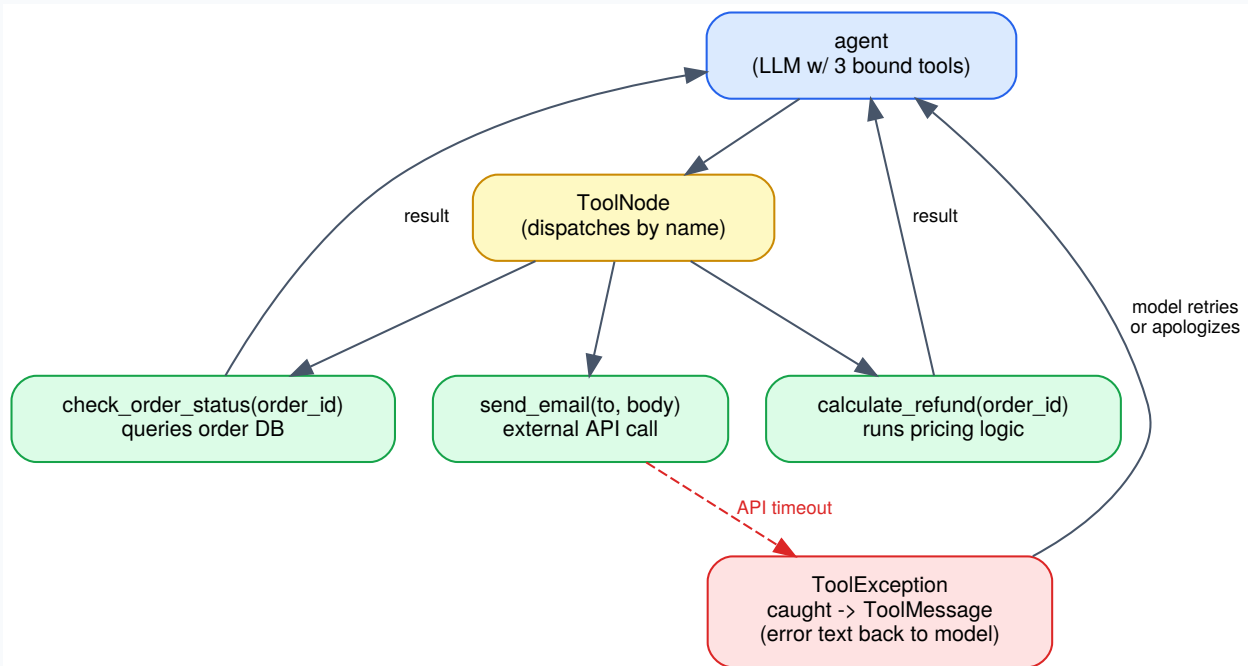
A: The right number of iterations is data-dependent: a narrow factual question might need one search, while a broad comparative question might genuinely need several rounds to gather enough distinct evidence. Hard-coding a fixed count either wastes calls on easy questions or under-researches hard ones — the entire point of an agentic loop is letting the model's own assessment of sufficiency drive the iteration count, bounded by a safety ceiling rather than replaced by a fixed one.

Tool Calling in LangGraph

6.1 Conceptual Overview

So far our agents have had one toy tool. Real agents are defined by the tools they have access to — and real tools fail, take real API calls, and have real side effects (sending an email can't be "undone" the way re-running a pure function can). This chapter is about handling that reality.

Running example — extending the support agent: Instead of a hard-coded `handle_refund` node, imagine the agent has three real tools it can call itself: `check_order_status(order_id)` (queries an orders database), `calculate_refund(order_id)` (runs business pricing logic), and `send_email(to, body)` (hits a third-party email API that can time out). The model decides which of these to call and in what order — your job is making sure a flaky `send_email` call doesn't crash the whole graph.



Multiple real tools behind one ToolNode, with error handling routed back to the model instead of crashing the graph

6.1.1 Binding multiple tools

You bind a list of tools to the model with `.bind_tools([...])`; the model sees all of their names, descriptions, and argument schemas, and can emit zero, one, or several tool calls in a single response. `ToolNode` dispatches each one by matching the tool call's `name` field to the corresponding tool function — you don't write that matching logic yourself.

6.1.2 Tool descriptions are part of your prompt engineering

The docstring on a `@tool`-decorated function isn't just documentation — it's what the model reads to decide whether and how to call that tool. A vague docstring like `"checks orders"` gives the model far less to go on than `"Look up the current shipping status and estimated delivery date for a given order ID. Use this before offering a refund, to confirm the order actually hasn't arrived."` — the second version tells the model both what the tool does and when to prefer it.

6.1.3 Handling tool failures without crashing the graph

A network timeout inside `send_email` is not a graph bug — it's an expected real-world event. `ToolNode` catches exceptions raised inside a tool by default and turns them into a `ToolMessage` containing the error text, which flows back to the model just like a normal tool result. This lets the model react intelligently (retry, apologize, try a different approach) instead of the whole `.invoke()` call raising an unhandled exception.

Key idea: Treat tool errors as *information for the model*, not as program crashes. An agent that sees "email API timed out" as a tool result can decide to retry or tell the user; an agent whose graph just throws an exception can do neither.

6.2 Implementation

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

@tool
def check_order_status(order_id: str) -> str:
    """Look up shipping status and estimated delivery date for an order.
    Use this before offering a refund, to confirm the order actually hasn't arrived."""
    # real implementation: query the orders database
    return "Order #4471: in transit, estimated delivery 5 days overdue."

@tool
def calculate_refund(order_id: str) -> str:
    """Calculate the refund amount owed for a given order, including any
    shipping fees, based on current store refund policy."""
    return "Refund amount: $42.99 (full price + shipping)."
```

```
@tool
def send_email(to: str, body: str) -> str:
    """Send a confirmation email to the customer. May fail if the email
    provider is unreachable – the caller should handle that gracefully."""
    import random
    if random.random() < 0.3:
        raise TimeoutError("Email provider did not respond in time.")
    return f"Email sent to {to}."
```

```
tools = [check_order_status, calculate_refund, send_email]
llm = ChatOpenAI(model="gpt-4o").bind_tools(tools)

class SupportState(TypedDict):
    messages: Annotated[list, add_messages]

def agent_node(state: SupportState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

# handle_tool_errors=True (the default) converts exceptions into
# ToolMessages instead of propagating them and killing the run
tool_node = ToolNode(tools, handle_tool_errors=True)

def should_continue(state: SupportState) -> str:
    return "tools" if state["messages"][-1].tool_calls else END

builder = StateGraph(SupportState)
builder.add_node("agent", agent_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue)
builder.add_edge("tools", "agent")

graph = builder.compile()

graph.invoke({
    "messages": [("user", "Order #4471 never arrived, please refund me and email confirmation.")]
})
# Model will typically call check_order_status -> calculate_refund -> send_email
# in sequence across loop iterations. If send_email raises TimeoutError,
# the model sees the error text and can apologize or suggest an alternative
# instead of the whole invoke() call crashing.
```

Note: For tools with side effects you truly cannot let the model call automatically without oversight — `send_email` to an external customer is a good candidate — Chapter 10 (Human-in-the-Loop) shows how to insert an approval gate *before* that specific tool executes, rather than trusting the model's judgment alone.

6.3 Interview Questions

Q1. Why does the docstring on a `@tool` -decorated function matter functionally, not just for documentation?

A: The docstring is sent to the LLM as the tool's description alongside its argument schema — it's literally what the model reads to decide whether to call that tool and with what reasoning. A vague or missing docstring gives the model poor grounds to choose correctly between similar tools (e.g., confusing `check_order_status` with `calculate_refund`), while a specific one ("use this before offering a refund...") actively guides correct sequencing.

Q2. What does `ToolNode(tools, handle_tool_errors=True)` actually change about how exceptions are handled?

A: By default, if a tool function raises an exception, `ToolNode` catches it and returns a `ToolMessage` containing the error text, rather than letting the exception propagate up and crash the entire `.invoke()` call. This means a flaky `send_email` timeout becomes information the model can act on (retry, inform the user) instead of an unhandled runtime error.

Q3. If the model emits two tool calls in a single response (e.g., `check_order_status` and `calculate_refund` together), how does `ToolNode` handle that?

A: `ToolNode` executes all requested tool calls from the last AI message — it isn't limited to one call per pass — and returns a `ToolMessage` for each, matched back to its originating `tool_call_id`. This lets a single agent turn fan out to multiple tool invocations before control returns to the model.

Q4. Why might you deliberately choose *not* to let a model call `send_email` automatically, even with error handling in place?

A: Error handling protects against the tool *failing*, but doesn't protect against the tool *succeeding when it shouldn't have* — e.g., the model misjudging the refund amount and emailing a customer an incorrect confirmation. For externally visible, hard-to-undo side effects, you generally want a human approval step before execution regardless of how reliable the tool itself is, which is a human-in-the-loop concern rather than an error-handling one.